



Ingeniería Ejecución en Computación e informática

Estructura de datos

Clase 15: TAD Árbol Binario
de Búsqueda

Mg. Francisco Philip Vásquez Iglesias



Resultados de aprendizaje

- 1.- Utilizar la notación asintótica para comparar algoritmos y determinar su calidad.
- 2.- Implementar estructuras de datos lineales para situaciones específicas en base a consideraciones de eficiencia de tiempos de acceso, utilización de recursos, tipos de implementación y complejidad.
- 3.- Implementar estructuras de datos no lineales para situaciones específicas en base a consideraciones de eficiencia de tiempos de acceso, utilización de recursos, tipos de implementación y complejidad.**

Árbol Binario de Búsqueda

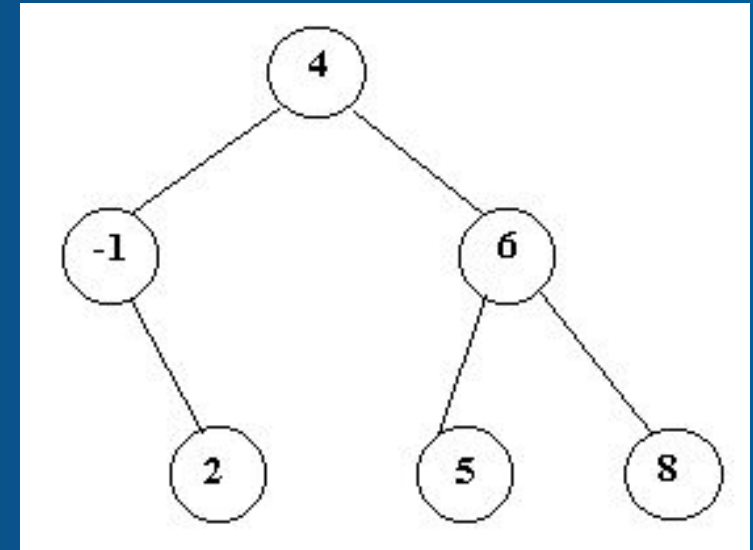


Árbol Binario de Búsqueda

- Un árbol binario de búsqueda es aquel que es compuesto por:
 - Una estructura vacía o...
 - Un elemento o clave de información más un número finito, **máximo dos**, de estructuras de tipo árbol, **disjuntos**, llamados subárboles y además cumplen con:
 - Todas las claves del subárbol izquierdo al nodo son menos que la clave del nodo.
 - Todas las claves del subárbol derecho al nodo son mayores a la clave del nodo.
 - Ambos subárboles son árboles binarios de búsqueda.

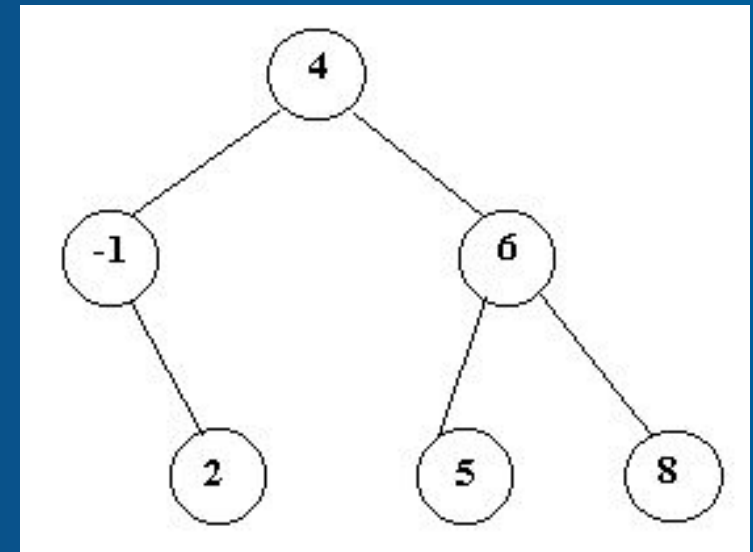
Ejemplo de árbol binario de búsqueda

- El ejemplo consiste en un árbol binario de búsqueda de números enteros.
- Dada la raíz “4”:
 - Las claves del subárbol izquierdo son menores que 4.
 - Las claves del subárbol derecho son mayores que 4.
- Esto se cumple también para ***todos los subárboles.***



Ejemplo de árbol binario de búsqueda

- ¿Qué tipo de recorrido entregaría las claves ordenadas de menor a mayor?.
- ¿Qué tipo de recorrido entregaría las claves ordenadas de mayor a menor?.
- Una ventaja fundamental de los árboles de búsqueda es que ***son en general mucho más rápidos*** para localizar un elemento que una lista enlazada.

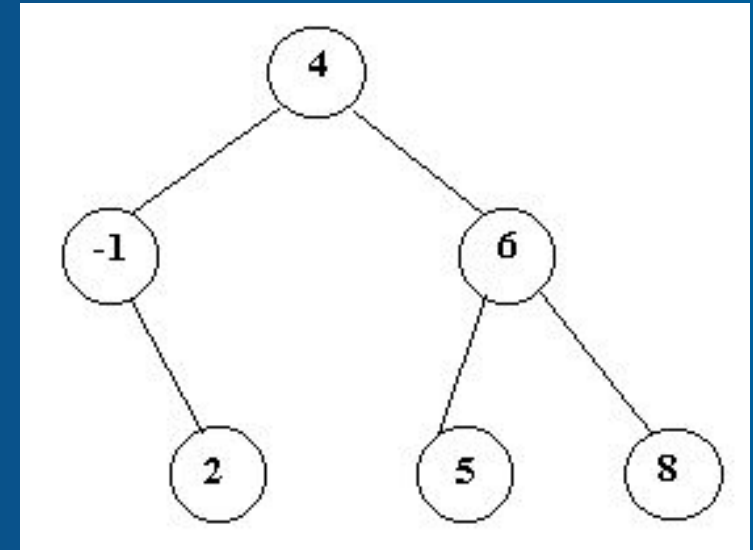


Árbol perfectamente equilibrado

- Se realiza analizando cada uno de los nodos del árbol.
- Significa que la diferencia entre el número de nodos del subárbol izquierdo y derecho es como máximo -1 o 1.
- Si se cumple lo anterior, el número de comparaciones necesarias para localizar una clave es aproximadamente de $\log_2(n)$ en el peor de los casos.

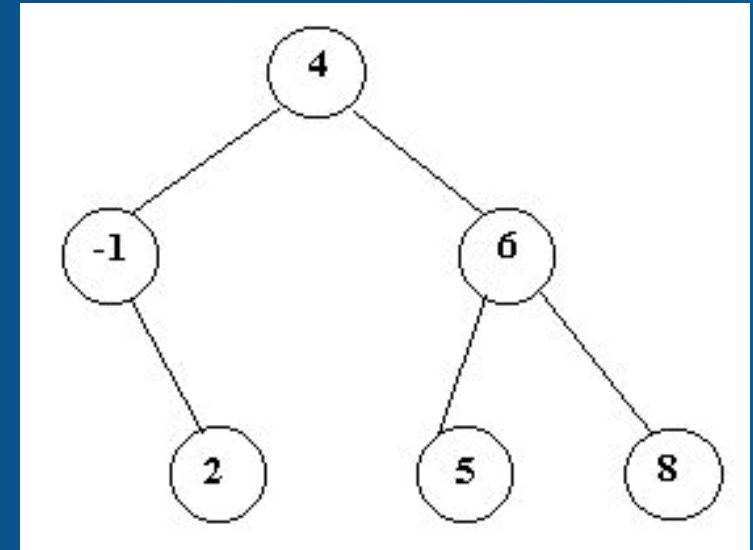
Inserción de elementos a un árbol binario de búsqueda

- El algoritmo de inserción para este tipo de árboles tiene la ventaja, sobre los arreglos ordenados, donde se emplearía búsqueda binaria, de que no necesita hacer una reubicación de los elementos de la estructura para que este siga ordenada.



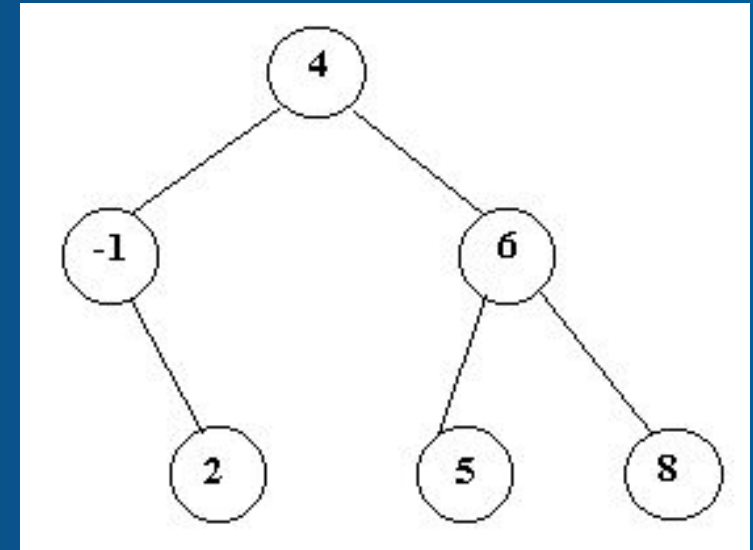
Insertión de elementos a un árbol binario de búsqueda

- Por ejemplo:
 - Insertar la clave 7 en el árbol se requiere avanzar por el árbol hasta llegar a la clave 8, e introducir la nueva clave en el subárbol izquierdo a 8.



Eliminado de elementos a un árbol binario de búsqueda

- Es un poco más complejo que el proceso de inserción, sin embargo, es más eficiente que el borrado de un arreglo ordenado.



Borrado

- El proceso de borrado de un árbol binario de búsqueda es complicado.
- Hay que comprobar que el árbol siga manteniendo sus características luego del borrado.
- Existen tres casos:
 1. El nodo no tiene descendientes.
 2. El nodo tiene descendientes por una rama.
 3. El nodo tiene descendientes en ambas ramas.



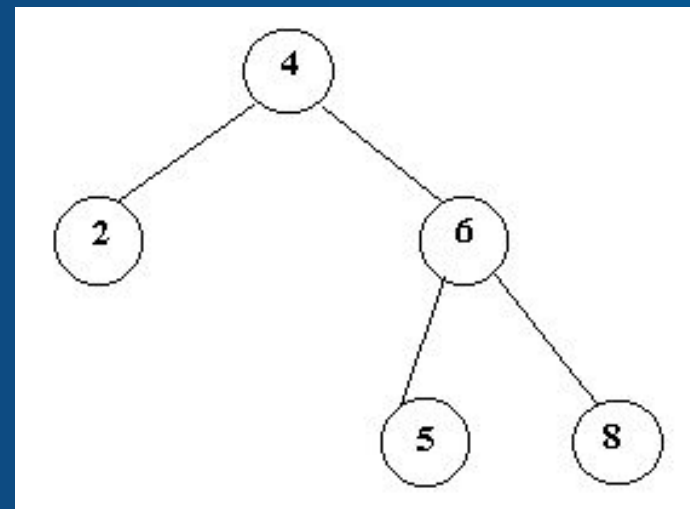
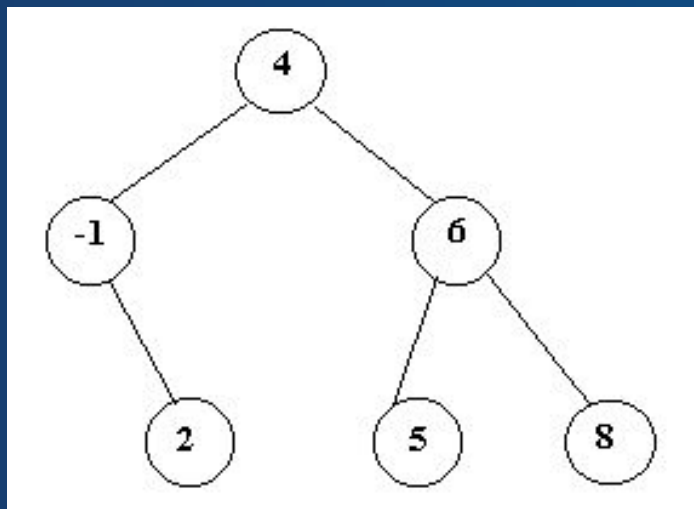
El nodo no tiene descendientes

- Simplemente se borra.

El nodo tiene descendiente por una rama

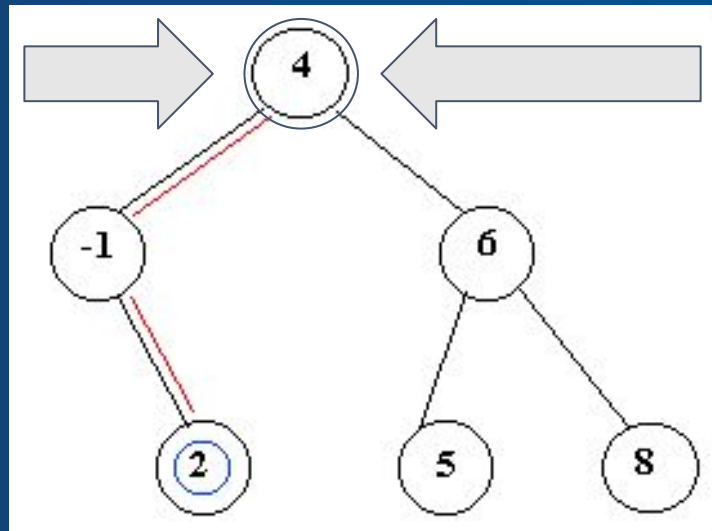
Si el nodo tiene al menos un descendiente por ***una sola rama***, se borra dicho nodo y su primer descendiente se asigna como hijo del padre del nodo borrado.

En el ejemplo se puede ver qué ocurre si se borra el nodo -1.



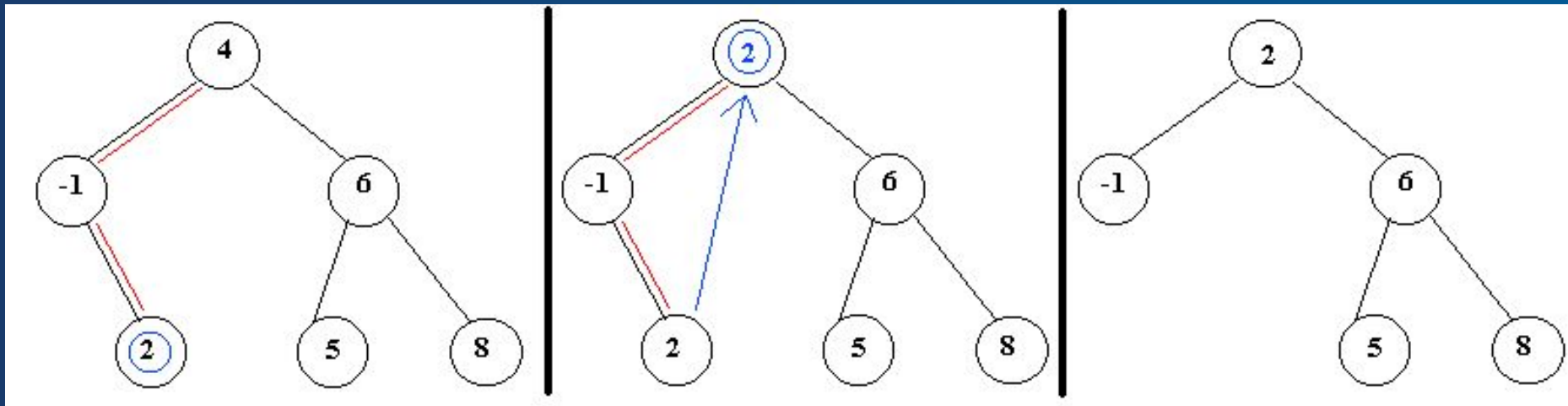
El nodo tiene al menos un descendiente por cada rama

- Al borrar un nodo, es necesario mantener la coherencia de los enlaces, además de mantener la estructura de árbol binario de búsqueda.
- La solución consiste en sustituir la información del nodo que se borra por el de una de las hojas, y borrar a continuación dicha hoja.



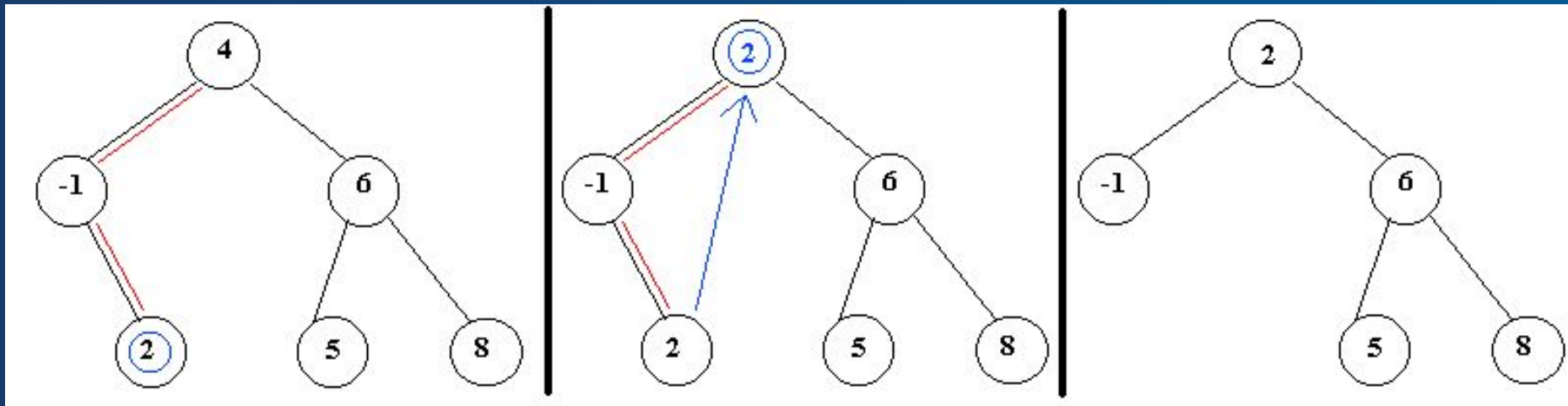
El nodo tiene al menos un descendiente por cada rama

- La hoja elegida debe poseer alguna de las siguientes claves:
 - La **mayor** de las claves **menores** al nodo que se borra. Si se quiere borrar el nodo 4, se sustituirá por el nodo de la clave 2.
 - La **menor** de las claves **mayores** al nodo que se borra. si se quiere borrar el nodo 4, se sustituirá por el nodo de la clave 5.



El nodo tiene al menos un descendiente por cada rama

- El ejemplo anterior realiza la sustitución por la mayor de las claves menores.
- Se desciende primer a la izquierda del nodo que se va a borrar, y después se avanza siempre a la derecha hasta encontrar un nodo hoja.



Ejemplo de creación de un Árbol Binario de Búsqueda



Ejemplo de creación de un Árbol Binario de Búsqueda

- Suponer que introduciremos las claves de forma ascendente:
 - Ejemplo: 1, 2, 3, 4, 5 y 6.
- Los pasos serían:
 1. Se crea un árbol cuya raíz tiene la clave 1.
 2. Se inserta la clave 2 en el subárbol derecho de 1.
 3. Se inserta la clave 3 en el subárbol derecho de 2.
 4. Continuando las inserciones, se ve que el árbol degenera en una lista secuencial.

Ejercicio en clases 1

- Almacenar en un árbol de búsqueda binaria los elementos del conjunto N , que son ingresados como sigue:
 - $N = \{10, 9, 1, 2, 4, 5, 3, 8, 7, 6\}$

Ejercicio en clases 2

- Almacenar en un árbol de búsqueda binaria los elementos del conjunto N, que son ingresados como sigue:
 - {50, 30, 70, 20, 40, 60, 80, 10, 35, 45, 65, 75, 85}

Ejercicio en clases 3

- Dado el siguiente conjunto de claves, construye un árbol binario de búsqueda insertando los elementos en el orden dado:
 - {50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45, 55, 65, 75, 90}
- Construye el árbol binario de búsqueda a partir de los datos dados.
- Realiza los siguientes recorridos:
 - Preorden.
 - Inorden.
 - Postorden.
 - Por amplitud.



Operaciones básicas sobre árboles binarios de búsqueda

Búsqueda

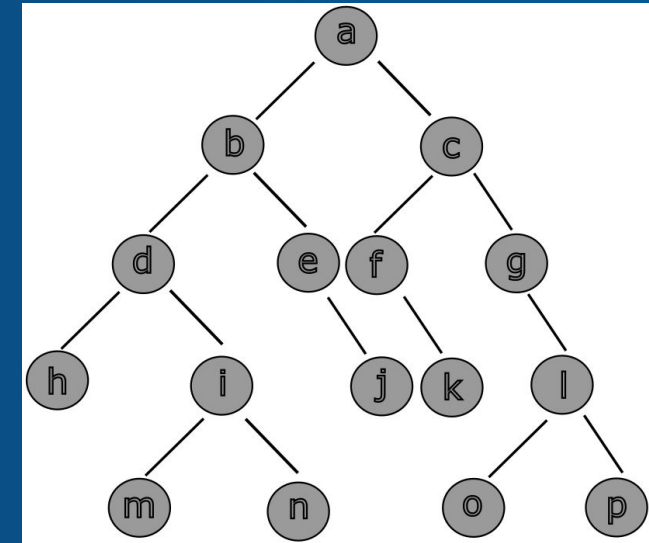
- Si el árbol no es de búsqueda, es necesario emplear uno de los recorridos anteriores sobre el árbol para localizarlo.
- El resultado es idéntico al de una búsqueda secuencial.
- Debido a las propiedades del árbol de búsqueda se puede acelerar la localización.
- Consiste en descender a lo largo del árbol de izquierda o derecha dependiendo del elemento buscado.

Función buscar en Árbol Binario de Búsqueda en C

```
bool buscar(arbolBin_t *a, char elem){
    if (a == NULL){
        printf("No se ha encontrado el elemento: %c",elem);
        return false;
    }
    else if (a->elem < elem)
        return buscar(a->der, elem);
    else if (a->elem > elem)
        return buscar(a->izq, elem);
    else
        printf("Se ha encontrado el elemento: %c",a->elem);
        return true;
}
```

La función no sirve para cualquier árbol binario

```
buscado='a';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);  
  
buscado='e';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);  
  
buscado='j';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);  
  
buscado='q';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);
```



```
Se inicia la busqueda del elemento a  
Se ha encontrado el elemento: a  
Se inicia la busqueda del elemento e  
No se ha encontrado el elemento: e  
Se inicia la busqueda del elemento j  
No se ha encontrado el elemento: j  
Se inicia la busqueda del elemento q  
No se ha encontrado el elemento: q
```


Inserción

- Esta operación resulta similar a la búsqueda.
- Cuando se llega a un árbol vacío, se crea el nodo en el puntero que se pasa por referencia.
- Si el elemento a insertar ya existe, entonces no hace nada.

```
void insertar(arbolBin_t **a, int elem){
    if (*a == NULL) {
        *a = (arbolBin_t *) malloc(sizeof(arbolBin_t));
        (*a)->elem = elem;
        (*a)->izq = (*a)->der = NULL;
    }
    else if ((*a)->elem < elem)
        insertar(&(*a)->der, elem);
    else if ((*a)->elem > elem)
        insertar(&(*a)->izq, elem);
}
```

Adaptación del código de Árbol Binario

```
arbolBin_t *a;  
char buscado;  
char abecedarioIncom[] = "abcdefghijklmnop";  
  
a=NULL;  
for(int i=0;i<16;i++){  
    insertar(&a, abecedarioIncom[i]);  
}
```

```
printf("\nSe inicia el recorrido en preorden:\n");  
preorden(a);  
printf("\nSe inicia el recorrido en inorden\n");  
inorden(a);  
printf("\nSe inicia el recorrido en postorden\n");  
postorden(a);  
printf("\nSe inicia el recorrido en amplitud\n");  
amplitud(a);
```



```
Se inicia el recorrido en preorden:  
a b c d e f g h i j k l m n o p  
Se inicia el recorrido en inorden  
Se inicia el recorrido en postorden  
p o n m l k j i h g f e d c b a  
Se inicia el recorrido en amplitud  
a b c d e f g h i j k l m n o p
```


Ejecución de la búsqueda

```
buscado='a';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);  
  
buscado='e';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);  
  
buscado='j';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);  
  
buscado='q';  
printf("\nSe inicia la busqueda del elemento %c\n",buscado);  
buscar(a, buscado);
```



```
Se inicia la busqueda del elemento a  
Se ha encontrado el elemento: a  
Se inicia la busqueda del elemento e  
Se ha encontrado el elemento: e  
Se inicia la busqueda del elemento j  
Se ha encontrado el elemento: j  
Se inicia la busqueda del elemento q  
No se ha encontrado el elemento: q
```

Función borrar nodo en C

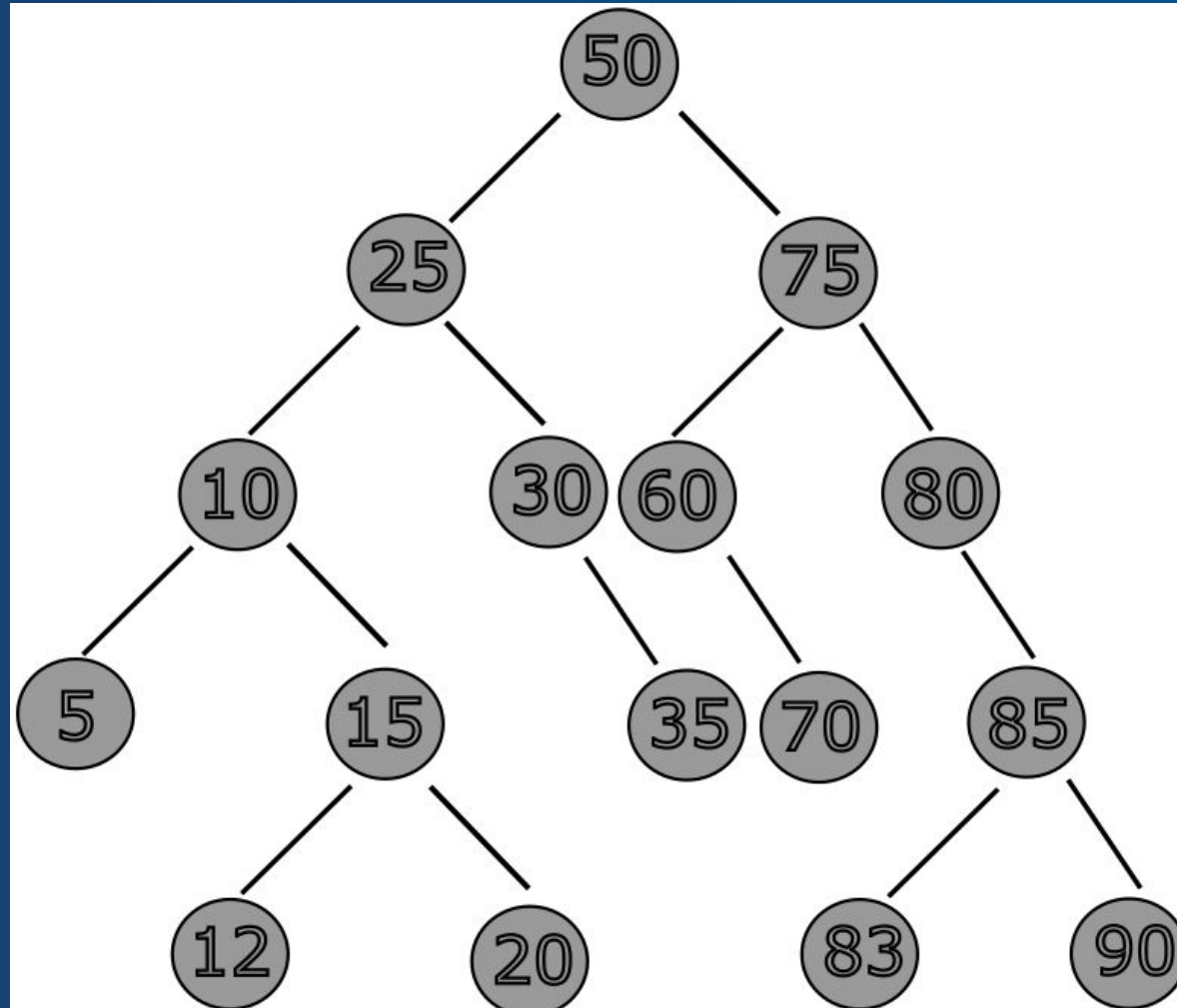
```
void borrar(arbolBin_t **a, int elem){
    arbolBin_t *aux;
    if (*a == NULL)
        return;
    if ((*a)->elem < elem)
        borrar(&(*a)->der, elem);
    else if ((*a)->elem > elem)
        borrar(&(*a)->izq, elem);
    else if ((*a)->elem == elem) {
        aux = *a;
        if ((*a)->izq == NULL)
            *a = (*a)->der;
        else if ((*a)->der == NULL)
            *a = (*a)->izq;
        else
            sustituir(&(*a)->izq, &aux);
        free(aux);
    }
}
```

```
//Sustituye el nodo borrado por el mayor de las menores
void sustituir(arbolBin_t **a, arbolBin_t **aux) {
    if ((*a)->der != NULL) {
        sustituir(&(*a)->der, aux);
    } else {
        (*aux)->elem = (*a)->elem;
        *aux = *a;
        *a = (*a)->izq;
    }
}
```

Ejemplo

Ejemplo

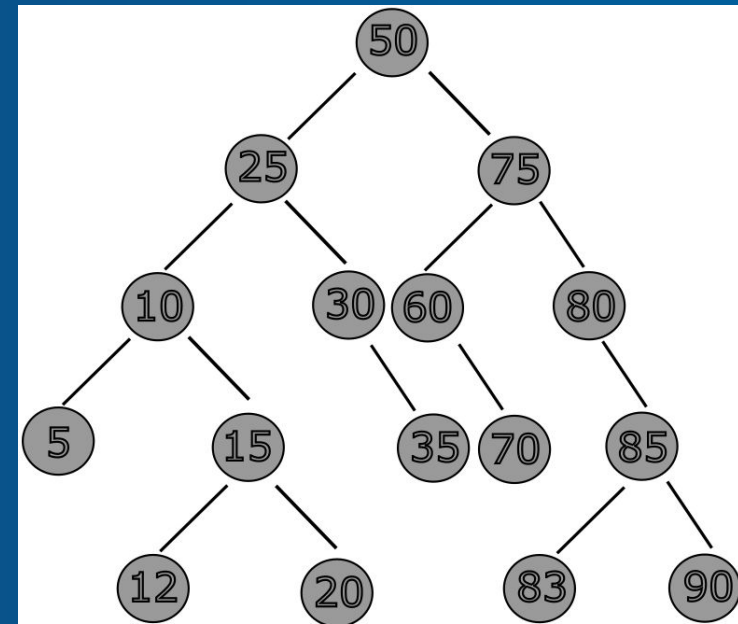
Ejemplo: Crear el siguiente árbol, recorrerlo y luego eliminar elementos



Ejemplo

```
arbolBin_t *a;  
int buscado, borrado;  
int array[] = {50,25,75,10,30,60,80,5,15,35,70,85,12,20,83,90};  
  
a=NULL;  
for(int i=0;i<16;i++){  
    insertar(&a, array[i]);  
}
```

```
printf("\nSe inicia el recorrido en preorden:\n");  
preorden(a);  
printf("\nSe inicia el recorrido en inorden\n");  
inorden(a);  
printf("\nSe inicia el recorrido en postorden\n");  
postorden(a);  
printf("\nSe inicia el recorrido en amplitud\n");  
amplitud(a);
```



```
Se inicia el recorrido en preorden:  
50 25 10 5 15 12 20 30 35 75 60 70 80 85 83 90  
Se inicia el recorrido en inorden  
5 10 12 15 20 25 30 35 50 60 70 75 80 83 85 90  
Se inicia el recorrido en postorden  
5 12 20 15 10 35 30 25 70 60 83 90 85 80 75 50  
Se inicia el recorrido en amplitud  
50 25 75 10 30 60 80 5 15 35 70 85 12 20 83 90
```

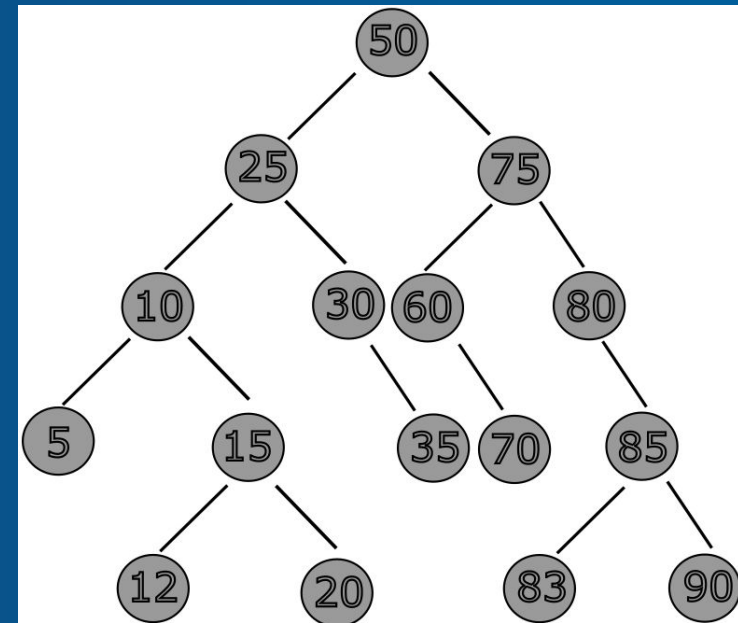
Ejemplo

```
buscado=50;
printf("\nSe inicia la busqueda del elemento %d\n",buscado);
buscar(a, buscado);

buscado=30;
printf("\nSe inicia la busqueda del elemento %d\n",buscado);
buscar(a, buscado);

buscado=100;
printf("\nSe inicia la busqueda del elemento %d\n",buscado);
buscar(a, buscado);

borrado=50;
printf("\nSe procede a eliminar el elemento %d\n",borrado);
borrar(&a, borrado);
buscado=50;
printf("\nSe inicia la busqueda del elemento %d\n",buscado);
buscar(a, buscado);
printf("\nSe inicia el recorrido en amplitud\n");
amplitud(a);
```



Se inicia la busqueda del elemento 50
Se ha encontrado el elemento: 50
Se inicia la busqueda del elemento 30
Se ha encontrado el elemento: 30
Se inicia la busqueda del elemento 100
No se ha encontrado el elemento: 100
Se inicia la busqueda del elemento 50
No se ha encontrado el elemento: 50
Se inicia el recorrido en amplitud
35 25 75 10 30 60 80 5 15 70 85 12 20 83 90

Ejercicio

EFES

Ejercicios

- Crear funciones que permitan obtener de nuestro árbol:
 - antecesor de un nodo.
 - sucesores de un nodo.
 - grado de un nodo.
 - Hojas del árbol.
 - Es nodo interno.
 - Nivel de un nodo.
 - Altura del árbol.
 - Anchura del árbol.



Ingeniería Civil informática

Estructura de datos

Fin de la clase

Mg. Francisco Philip Vásquez Iglesias

